



Advancement Report

AI-Powered Intrusion Detection System for Cloud Environments

KRID Aboubakr

Master of Artificial Intelligence and Cybersecurity

Supervised by Prof. Fakhri Yousef

April 2026

Contents

1	Project Understanding	2
2	Existing Approaches	2
2.1	Signature-Based Systems	2
2.2	Commercial Cloud-Native Security Tools	3
2.3	Academic Machine Learning Approaches	3
2.3.1	Classical Machine Learning Approaches	5
2.3.2	Deep Learning for Intrusion Detection	5
2.3.3	Advanced and Emerging Techniques	6
2.3.4	Summary of Academic Approaches	7
3	Project Flow	7
3.1	Overview	7
3.2	Data Ingestion	7
3.3	Data Processing	9
3.4	Training Pipeline	9
3.5	Detection and Response	9
4	Proposed Approach	9
4.1	General Strategy	9
4.2	Core Implementation	10
4.3	Personal Contribution	10
	References	11

1. Project Understanding

The goal of this project is to design and implement an **Intrusion Detection System (IDS)** adapted to cloud environments. The system analyses network traffic and system logs, applies machine learning to identify anomalous or malicious behaviour, and generates alerts for the security team. The project covers the full pipeline: from raw data collection to model training, real-time detection, and result visualisation.

The subject is relevant both academically and practically. Cloud-native security tools offered by major providers are closed, vendor-specific, and unavailable to organisations operating private or hybrid clouds. A custom, open, and reproducible IDS directly addresses this gap, and constitutes a concrete object of study at the intersection of machine learning and cybersecurity.

2. Existing Approaches

Before defining our own approach, it is essential to survey what already exists in the field, understand why each category of solution falls short in specific contexts, and identify the gap this project aims to fill.

2.1. Signature-Based Systems

The oldest and most widely deployed intrusion detection approach is **signature-based detection**, exemplified by tools such as Snort and Suricata. These systems maintain a database of known attack patterns and flag any traffic that matches one of these signatures.

The fundamental limitation is well-documented in the literature. Lunt1992 and later empirical studies confirm that a signature-based system cannot detect any attack for which no signature has been written. A study published by Holm (2014) at IEEE, which tested 356 attacks against Snort configured with an official rule set, found that the tool's detection rate for zero-day attacks (attacks unknown to the rule set) was only **8.2%** in conservative estimates [1]. For known attacks, the detection rate rose to 54%, underscoring how dependent these systems are on prior knowledge.

A separate review in the Cybersecurity journal confirms the structural cause of this limitation: *“SIDS has difficulty in detecting zero-day attacks for the reason that no matching signature exists in the database until the signature of the new attack is*

extracted and stored” [2]. Additionally, writing and maintaining rule sets requires significant expert effort, and signatures must be continuously updated as new attack variants emerge [4].

2.2. Commercial Cloud-Native Security Tools

Major cloud providers offer their own built-in threat detection services: **AWS GuardDuty**, **Microsoft Defender for Cloud**, and **Google Cloud Security Command Center**. These tools use a combination of rule-based detection and proprietary machine learning applied to cloud-specific log sources such as VPC Flow Logs, CloudTrail API events, and DNS queries.

While effective within their respective ecosystems, these solutions carry three limitations that are relevant to this project:

- **Vendor lock-in.** Each tool operates exclusively within its cloud provider’s platform. An organisation running a private cloud (OpenStack, VMware) or a multi-cloud setup cannot deploy these tools at all.
- **Opacity.** These services do not expose their detection logic. AWS itself acknowledges false positives in GuardDuty and provides suppression rules specifically to help users filter them [5]. However, the root cause of a false positive cannot be inspected or corrected, as the underlying model is inaccessible.
- **Operational noise.** Practitioners report that GuardDuty generates findings for legitimate activity such as routine DNS lookups, NAT gateway traffic, and CI/CD pipeline IP addresses, requiring significant manual tuning after deployment [6].

2.3. Academic Machine Learning Approaches

The academic community has extensively explored Machine Learning (ML) and Deep Learning (DL) to overcome the inherent limitations of signature-based and closed commercial IDS [2, 4]. These approaches can be broadly categorized into several key paradigms, as outlined in Table 1.

Table 1: A Taxonomy of ML/DL Approaches in Academic IDS Research

Approach	Cate- gory	Key Techniques	Strengths & Limitations
Classical	Ma- chine Learning	Decision Trees (DT), Random Forest (RF), Support Vector Machines (SVM), k-Nearest Neighbors (k-NN)	High performance on known attacks; limited ability to detect zero-day threats without feature engineering [2].
Deep Learning for Supervised Learning		Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM)	Automatic feature extraction, excels at learning complex patterns; computationally intensive and requires large labeled datasets [10].
Deep Learning for Unsupervised Learning		Autoencoders (AE), Variational Autoencoders (VAE), Generative Adversarial Networks (GAN)	Effective for anomaly and zero-day attack detection without labeled data; can suffer from high false positive rates [13].

Continued on next page

Table 1 – continued from previous page

Approach	Cate- gory	Key Techniques	Strengths & Limitations
Advanced and Emerging Techniques	and Tech-	Transformers, Large Language Models (LLMs), Ensemble Methods, Federated Learning, Few-Shot/Zero-Shot Learning	State-of-the-art in capturing complex relationships and addressing data scarcity; challenges remain in interpretability and computational cost [9, 8].

2.3.1 Classical Machine Learning Approaches

Early academic efforts focused on adapting classical ML algorithms for intrusion detection. Algorithms such as Decision Trees (DT), Random Forest (RF), Support Vector Machines (SVM), and k-Nearest Neighbors (k-NN) have been widely applied to classify network flows as benign or malicious based on a set of engineered features [2, 4]. While these models can achieve high accuracy on benchmark datasets for known attack types, their performance is heavily dependent on the quality of feature engineering and they often struggle to generalize to new, unseen (zero-day) attacks [2]. Their relative simplicity, however, makes them strong candidates for establishing a baseline performance in any new IDS development, as is planned for this project.

2.3.2 Deep Learning for Intrusion Detection

The shift towards Deep Learning (DL) in IDS research was motivated by its ability to automatically learn hierarchical feature representations from raw data, reducing the need for manual feature engineering [10]. Research in this area can be divided into supervised and unsupervised approaches.

Supervised Deep Learning Models Supervised DL models are trained on large labeled datasets to classify network traffic. Common architectures include Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), which are adept at

extracting spatial features from data, and Recurrent Neural Networks (RNN) like Long Short-Term Memory (LSTM) networks, which excel at capturing temporal dependencies in sequential data like network flows [10]. For instance, a hybrid CNN-LSTM model was shown to effectively leverage both spatial and temporal patterns, achieving high detection rates on benchmark datasets like CICIDS-2017 [11]. Despite their power, these models require substantial computational resources and large, well-labeled datasets for effective training [14].

Unsupervised and Generative Deep Learning Models Unsupervised learning methods, particularly Autoencoders (AE) and Generative Adversarial Networks (GAN), have gained prominence for their ability to detect anomalies without requiring labeled attack data [12]. Autoencoders learn a compressed representation of "normal" network traffic and can identify intrusions as deviations from this norm (i.e., high reconstruction error) [10]. GANs, consisting of a generator and discriminator, can be used to model normal behavior and detect anomalies that do not fit the learned distribution [13]. These methods are particularly valuable for identifying novel and zero-day attacks, a key weakness of signature-based systems. A 2024 review of unsupervised methods highlighted their potential but also noted that they often produce higher false positive rates compared to their supervised counterparts [12].

2.3.3 Advanced and Emerging Techniques

The academic frontier is pushing beyond traditional DL architectures to address specific challenges in IDS.

Transformers and Large Language Models (LLMs) Inspired by their success in Natural Language Processing (NLP), Transformers and LLMs are being adapted for IDS due to their powerful attention mechanisms, which can effectively model long-range dependencies in sequential data [9]. A 2024 survey by Kheddar et al. provides a comprehensive analysis of how architectures like BERT, GPT, and Vision Transformers (ViT) are being applied to cyber-threat detection, including in cloud environments [9]. These models show promise in capturing complex attack patterns but pose significant challenges in terms of computational cost and interpretability [9, 14].

Ensemble Methods Ensemble learning techniques, such as Random Forest (already a classic approach) and boosting algorithms like XGBoost, combine multiple models to improve overall prediction accuracy and robustness. A 2024 survey on ensemble methods in IDS confirms that they are frequently used to enhance detection performance by leveraging the strengths of various base learners [8]. These

methods often represent the state-of-the-art in benchmark competitions, providing a high-performance target for new models.

Addressing Data Scarcity: Few-Shot and Zero-Shot Learning A major practical challenge in IDS is the scarcity of labeled data for new or rare attacks. Techniques like few-shot and zero-shot learning are being explored to mitigate this. For example, the NERO framework uses metric-based meta-learning to detect zero-day attacks in IoT environments with limited training data [7]. These approaches are highly relevant for cloud environments where new attack vectors constantly emerge.

2.3.4 Summary of Academic Approaches

The academic landscape is rich with ML and DL approaches that directly address the shortcomings identified earlier. These methods, summarized in Table 1, provide the technical foundation for building a modern, AI-powered IDS. While powerful, many of these advanced models function as "black boxes," limiting the ability of security analysts to understand and trust their decisions. This underscores the importance of the explainability layer proposed in this report, which aims to bridge this critical gap between academic innovation and practical, actionable security.

3. Project Flow

3.1. Overview

The system is organised around four interconnected blocks that cover the full pipeline from raw cloud data to security alerts. Figure 1 presents the overall architecture, displayed on the following page.

3.2. Data Ingestion

The pipeline begins with the **Data Ingestion** block. A Traffic Collector captures raw network packets and events generated by the cloud infrastructure. These are forwarded through a message queue (Log Streaming / MQ) which buffers incoming data and decouples the collection layer from the processing layer an important design choice that prevents data loss during traffic spikes. The raw events are persisted in a Raw Data Storage component for replay and reprocessing.

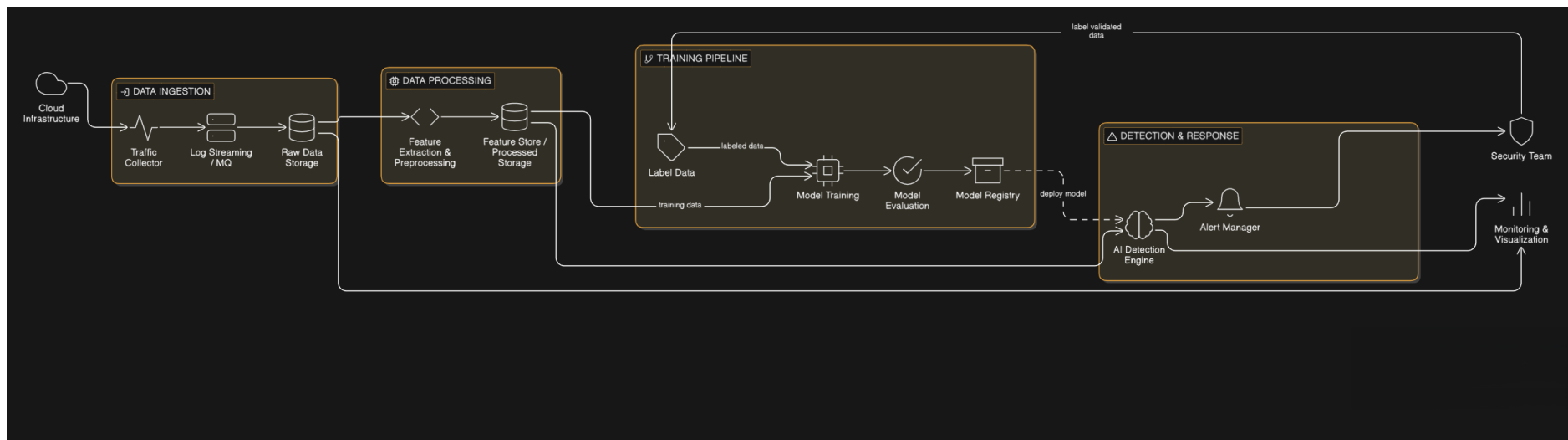


Figure 1: System architecture overview

3.3. Data Processing

The **Data Processing** block transforms raw events into structured feature vectors usable by a machine learning model. The Feature Extraction and Preprocessing step computes flow-level statistics from the raw data: packet counts, byte distributions, connection duration, port information, and protocol indicators. The resulting vectors are stored in a Feature Store, which acts as the shared input source for both the training pipeline and the live detection engine.

3.4. Training Pipeline

The **Training Pipeline** is responsible for producing and maintaining the detection model. Data from the Feature Store is first labelled using ground-truth attack categories derived from a public benchmark dataset. The labelled data feeds into a Model Training step, the resulting model is evaluated against a held-out test set, and if performance is acceptable it is registered in the Model Registry. A feedback loop is present in the architecture: validated labelled data can be reinjected into the training process, allowing the model to improve incrementally over time.

3.5. Detection and Response

Once a trained model is available, it is deployed to the **AI Detection Engine**, which receives the real-time feature stream from the Feature Store and classifies each incoming observation. Detections are routed to the Alert Manager, which filters and deduplicates alerts before forwarding them to two outputs: the Security Team for direct response, and the Monitoring and Visualisation dashboard for continuous operational overview.

4. Proposed Approach

4.1. General Strategy

The approach adopted for this project is deliberately pragmatic. The objective is to implement a complete and functional version of the pipeline described above, using established tools and well-understood techniques, and to contribute **one focused improvement** over a standard baseline.

This strategy is intentional. A project that replicates a known pipeline reliably and demonstrates it working end-to-end is more valuable than an ambitious proposal that remains partially implemented. The scope will be adjusted as the project progresses;

the architecture described here defines the target, not a committed specification.

4.2. Core Implementation

The system will follow the four-block architecture shown in Figure 1. The technology choices for each block are guided by maturity, documentation, and alignment with the project subject:

- **Data ingestion:** network traffic capture from the lab environment using standard packet capture tools, with a message queue to buffer incoming events.
- **Data processing:** feature extraction from raw flows using Python, producing a structured dataset compatible with the format of the CICIDS-2017 benchmark [3].
- **Model training:** a supervised classifier trained on the CICIDS-2017 labelled dataset. A baseline model (Random Forest or equivalent) will be trained first to establish a reference performance level before any further experimentation.
- **Detection and alerting:** the trained model applied to incoming feature vectors in near-real-time, with results displayed on a monitoring dashboard.

4.3. Personal Contribution

On top of the baseline pipeline, one specific contribution is planned to differentiate this project from a straightforward replication. The current direction is to add an **explainability layer** to the alert output: when the model flags a flow as malicious, the system will indicate which features contributed most to that decision. This addresses a practical limitation of most existing IDS implementations alerts that provide no context are difficult for an analyst to act on.

This contribution is modest in scope but meaningful in practice. It does not require a different model or architecture; it adds a post-processing step to the detection output. The exact method will be determined during the implementation phase.

References

- [1] H. Holm, “*Signature Based Intrusion Detection for Zero-Day Attacks: (Not) A Closed Chapter?*”, in *Proc. IEEE 47th Hawaii International Conference on System Sciences (HICSS)*, IEEE, 2014. <https://ieeexplore.ieee.org/document/6759203>
- [2] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “*Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges*”, *Cybersecurity*, vol. 2, no. 20, Springer, 2019. <https://doi.org/10.1186/s42400-019-0038-7>
- [3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “*Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*”, in *Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP)*, Portugal, January 2018. <https://www.unb.ca/cic/datasets/ids-2017.html>
- [4] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, “*Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches*”, *Transactions on Emerging Telecommunications Technologies*, Wiley, 2021. <https://doi.org/10.1002/ett.4150>
- [5] Amazon Web Services, “*Suppression Rules in GuardDuty*”, AWS Documentation, 2022. https://docs.aws.amazon.com/guardduty/latest/ug/findings_suppression-rule.html
- [6] AWS re:Post Community, “*GuardDuty False Positive Rates*”, AWS re:Post, March 2022. <https://repost.aws/questions/QUYWBwmzV4SF-W0g5j3r5eFg/guardduty-false-positive-rates>
- [7] J. F. Cevallos M., A. Rizzardi, S. Sicari, and A. Coen Porisini, “*NERO: NEural algorithmic reasoning for zeRO-day attack detection in the IoT: A hybrid approach*”, *Computers & Security*, 2024. <https://doi.org/10.1016/j.cose.2024.103898>
- [8] Anitha and R. Gandhi, “*Performance analysis of ensemble learning algorithms in intrusion detection systems: A survey*”, in *AIP Conference Proceedings*, vol. 2915, no. 1, 2024. <https://pubs.aip.org/aip/acp/article-abstract/2915/1/020017/3289129/Performance-analysis-of-ensemble-learning?redirectedFrom=PDF>

-
- [9] H. Kheddar, “*Transformers and Large Language Models for Efficient Intrusion Detection Systems: A Comprehensive Survey*”, arXiv preprint arXiv:2408.07583, 2024. <https://arxiv.org/abs/2408.07583>
- [10] R. Kimanzi, P. Kimanga, D. Cherori, and P. K. Gikunda, “*Deep Learning Algorithms Used in Intrusion Detection Systems – A Review*”, arXiv preprint arXiv:2402.17020, 2024. <https://api.semanticscholar.org/CorpusID:268032967>
- [11] Z. Li, W. Fang, C. Zhu, G. Song, and W. Zhang, “*Toward Deep Learning based Intrusion Detection System: A Survey*”, in *Proc. 6th Int. Conf. on Big Data Engineering*, 2024. <https://dl.acm.org/doi/pdf/10.1145/3688574.3688578>
- [12] N. Sharma, A. Thakur, and B. S. Sohi, N. Sharma, B. Arora, S. Ziyad, P. Kumar Singh and Y. Singh “*A Holistic review and performance evaluation of unsupervised learning methods for network anomaly detection*”, *International Journal on Smart Sensing and Intelligent Systems*, 2024. <https://doi.org/10.2478/ijssis-2024-0016>
- [13] S. R. Sindiramutty, K. R. V. Prabakaran, N. Z. Jhanjhi, R. K. Murugesan, S. N. Brohi, and M. Masud, “*Generative AI in Network Security and Intrusion Detection*”, in *Reshaping CyberSecurity With Generative AI Techniques*, IGI Global, 2024, pp. 77–124. <https://www.igi-global.com/chapter/generative-ai-in-network-security-and-intrusion-detection/356771>
- [14] F. M. Anis, M. Alabdullatif, S. Aljbli, and M. Hammoudeh, “*A Survey on the Applications of Deep Learning in Network Intrusion Detection Systems to Enhance Network Security*”, *IEEE Access*, 2025. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=11215731>